INFERENCE ENGINEERING
ASSIGNMENT

Name: Sahil Ranade

Assignment: Inference Engineering

1. Why is Inference Memory-Bandwidth Bound Important?

Answer

While running an LLM, the GPU has to perform a large number of calculations using the parameters that were learned during training. These parameters are normally kept in the GPU's VRAM so that they can be accessed during inference.

The important point is that performing calculations is only one part of the process. The GPU also has to continuously read the required model data from memory. If this data cannot be supplied quickly enough, the GPU's calculation units cannot work at their full potential.

This is the main reason why memory bandwidth is important in LLM inference.

Meaning of Memory Bandwidth

Memory bandwidth represents how much information can be transferred between memory and the processing units in a specific amount of time.

It is usually expressed in:

- GB/s – Gigabytes per second
- TB/s – Terabytes per second

It can be represented as:

$$\text{Memory Bandwidth} = \frac{\text{Data Transferred}}{\text{Time}}$$

For example, consider two GPUs that have similar computational capability but different memory bandwidths. The GPU with higher bandwidth can supply data to its compute units more quickly.

Why Does This Matter for LLMs?

LLMs generally generate text one token at a time. For every new token, the model performs calculations using its learned weights.

A simplified process is:

$$\text{Input} \rightarrow \text{Model Weights} \rightarrow \text{GPU Computation} \rightarrow \text{Next Token}$$

This process repeats until the complete response is generated.

If the GPU spends a significant amount of time waiting for model data to arrive from memory, the workload becomes **memory-bandwidth bound**.

Importance for a Company

For a company serving an LLM to many users, inference performance directly affects the user experience and operating expenses.

Memory bandwidth can influence:

- Token generation rate
- Response latency
- GPU utilization
- Number of requests handled simultaneously
- Cost of running the service

Therefore, when choosing a GPU for LLM inference, it is not enough to look only at its compute performance. Memory capacity and memory bandwidth are also important factors.

2. How Can We Calculate the Overall Memory Required to Store a Model? How Are LLMs Represented When Stored?

Answer

The memory required by an LLM mainly comes from the parameters that it has learned during training. A model with billions of parameters needs a large amount of storage, but the exact requirement also depends on how each parameter is represented.

The first thing to consider is the numerical precision used for storing the weights.

Common Parameter Formats

Format	Bits	Memory per Parameter
FP32	32	4 bytes
FP16	16	2 bytes
BF16	16	2 bytes
INT8	8	1 byte
INT4	4	0.5 byte

Since one byte contains eight bits, the approximate memory occupied by model weights can be calculated using:

$$\boxed{\text{Weight Memory} = \text{Parameter Count} \times \text{Bytes per Parameter}}$$

Example Calculation

Consider a model containing 7 billion parameters.

For FP32:

$$7 \times 10^9 \times 4 = 28 \times 10^9 \text{ bytes}$$

Therefore:

$$\boxed{\text{FP32} \approx 28 \text{ GB}}$$

For FP16:

$$7 \times 10^9 \times 2 = 14 \times 10^9 \text{ bytes}$$

Therefore:

$$\boxed{\text{FP16} \approx 14 \text{ GB}}$$

For the lower-precision formats:

$$\boxed{\text{INT8} \approx 7 \text{ GB}}$$

and

$$\boxed{\text{INT4} \approx 3.5 \text{ GB}}$$

This is why quantization can be useful when trying to run a large model on hardware with limited memory.

How LLM Parameters Are Organized

The parameters of an LLM are not stored as one continuous collection of unrelated values. They are arranged into structures called **tensors**.

A tensor is a multidimensional arrangement of numerical values.

A Transformer model can have tensors corresponding to different parts of the architecture, such as:

- Token embedding layers
- Attention projections
- Query, Key and Value parameters
- Output projections
- MLP or feed-forward layers
- Normalization parameters

For example, the attention mechanism commonly contains matrices represented by:

$$W_Q, \quad W_K, \quad W_V, \quad W_O$$

Each of these contains numerical values learned during training.

Model Files

When an LLM is saved, its tensors and their numerical values are stored in a model file along with information needed to load them.

Some commonly encountered model formats include:

- Safetensors
- GGUF

These formats can contain information such as tensor names, tensor shapes and data types along with the actual model data.

Storage Memory vs Inference Memory

The calculated weight size should not be treated as the total VRAM requirement during inference.

While running the model, additional memory can be required for:

- KV cache
- Activations
- Temporary computation data
- Runtime overhead
- Other framework-related allocations

Therefore:

$$\text{Total Inference Memory} > \text{Weight Memory}$$

in many practical situations.

For example, even if a model's weights occupy around 14 GB, a GPU with exactly 14 GB of VRAM may not have enough free memory to run it comfortably.

This is an important consideration when deciding whether a particular model can be deployed on a particular GPU.

3. Explain Each Layer of Inferencing and Its Importance to a Company

Answer

Running an LLM in a real application requires more than just the model itself. Different software and hardware components work together to receive requests, execute the model and monitor its performance.

I can understand the inference system through three major layers:

1. Runtime
2. Infrastructure
3. Tooling

Each layer solves a different problem.

3.1 Runtime

The runtime is the component that takes the model and actually executes it.

After a model has been trained, the runtime is responsible for making it usable for inference. It loads the model, performs the required operations and manages the generation process.

Examples include:

- vLLM
- TensorRT-LLM
- llama.cpp

A runtime may handle:

- Model loading
- Token generation
- Request scheduling
- Batching
- KV-cache management
- Interaction with GPUs

The main objective is to use the available hardware efficiently.

For example, if several users send requests at the same time, the runtime decides how those requests should be processed. Efficient scheduling can increase the amount of work completed by the GPU and reduce waiting time.

For a company, this can result in better throughput and lower inference cost without changing the trained model itself.

3.2 Infrastructure

Infrastructure represents the physical and system resources on which the inference service runs.

It can include:

- GPUs
- CPUs
- System memory
- Servers
- Storage
- Network connections
- Load balancers
- Multiple-server clusters

A small application might use one GPU, while a large company may require several GPU servers.

A basic large-scale setup can be represented as:

Users → Application → Load Balancer → GPU Servers → LLM

Infrastructure is not only about buying powerful GPUs. The company also has to make sure that the complete system can handle increasing numbers of users.

Important considerations include:

- GPU memory capacity
- Network performance
- Storage
- Power consumption
- Reliability

- Scaling
- Failure handling

A well-designed infrastructure allows the service to continue working even as demand increases.

3.3 Tooling

Tooling is used to understand what is happening inside the inference system.

Instead of manually guessing why a model is slow or consuming too much memory, engineers can use different tools to collect performance information.

Typical uses include:

- Benchmarking
- GPU profiling
- Latency measurement
- Throughput measurement
- Memory monitoring
- Logging
- Tracing
- Bottleneck analysis

For example, imagine that an application takes longer than expected to generate a response.

The engineer can investigate different parts of the system:

$$\text{GPU Compute} \rightarrow \text{Memory} \rightarrow \text{KV Cache} \rightarrow \text{Networking} \rightarrow \text{Scheduling}$$

This helps identify where the actual performance problem is located.

Tooling is therefore important because optimization should be based on measured performance rather than assumptions.

3.4 Comparison of the Three Layers

The three layers can be viewed as different questions asked while building an inference system.

Layer	Main Role
Runtime	Runs the model and manages inference requests efficiently.
Infrastructure	Provides the hardware and system environment required to serve users.
Tooling	Measures system behavior and helps find performance problems.

3.5 Complete View of an Inference System

The overall process can be visualized as:

User → Application → API → Runtime → GPU → Model

The infrastructure provides the hardware and system resources needed by these components, while tooling observes the system and provides performance information.

From a company perspective, these three layers are connected.

For example:

- A better runtime can improve GPU utilization.
- Better infrastructure can support more users.
- Better tooling can identify areas where optimization is required.

If any one of these areas is poorly designed, the complete inference service can become inefficient.

Importance to a Company

For a company, the final goal is not simply to make the model work. The system should also be able to handle users efficiently and economically.

Good inference engineering can help improve:

- Performance
- Scalability
- Reliability
- Response latency
- Resource utilization
- Cost efficiency

Therefore, runtime, infrastructure and tooling together form an important part of deploying an LLM in a real-world environment.

Conclusion

From these topics, I understood that LLM inference depends on both the model and the system used to run it.

Memory bandwidth is important because the GPU needs to continuously access model data while generating tokens. The memory requirement of a model can be estimated from its parameter count and the precision used to store those parameters, but actual inference also requires additional memory.

I also understood that an inference system has multiple parts. The runtime executes the model, infrastructure provides the required hardware and tooling helps engineers measure and improve the system.

So, efficient inference is important for companies because it can provide better performance while keeping resource usage and operating costs under control.